- static fields total 48 bytes
- `o_size` includes tombstones
- overflow slot 0 is not valid (null key, null value, next points to itself)
- `o_cap` includes slot 0, but `o_size` does not.
- `m_cap` must be ≥6
- `o_cap` must be ≥2
- all integers are unsigned

The first 6 buckets are always inlined so that the struct can have `[[gnu::aligned(64)]]` without GCC adding any extra padding.

When allocating, it allocates `sizeof(struct MapImpl) + sizeof(MapEntry) * (m_cap - 6)`.

The fields are ordered so the fields that need to be zeroed on creation (`m_size`, `o_size`, `o_tcnt`) are grouped at the start, so they can happen in one AVX2 write.

overflow is implemented as a separate allocation for simplicity, but it would be doable to put everything in the same allocation since resizing the map itself (`m_cap`) creates a new map and copies everything over, so there will never be a case where `Map_mresize` moves inline entries but not the overflow.

The number of live entries in the map can be calculated as: `m_size + o_size - o_tcnt`. The map structure is allocated together, rather than the variable struct member separately from the rest. The map API supports both owning and non-owning maps, though the ownership model has to be the same for all keys and values within any map, and it must be stored externally (notably, `Map_merge` has a mode where one of the ownerships being unowned can actually be a special case of owned, so it would be more difficult to keep the two case separate if the struct contained the ownership). structurally, the minimum values for `m_cap` and `o_cap` are 6 and 2, respectively, but the implementation sets them higher, at 11 and 4, respectively.

### `struct MapImpl` fields

| | | |
|---|---|---|
| 8-bytes | m_size | # of non-empty buckets |
| 8-bytes | o_size | # of overflow slots used |
| 8-bytes | o_tcnt | # of tombstones in overflow |
| 8-bytes | m_cap | # of total buckets |
| 8-bytes | o_cap | # of overflow slots |
| 8-bytes | overflow | pointer to overflow arena |
| 144-bytes | buckets | first 6 inline bucket entries |
| variable | _rest[] | the rest of the buckets |

### `MapEntry` fields

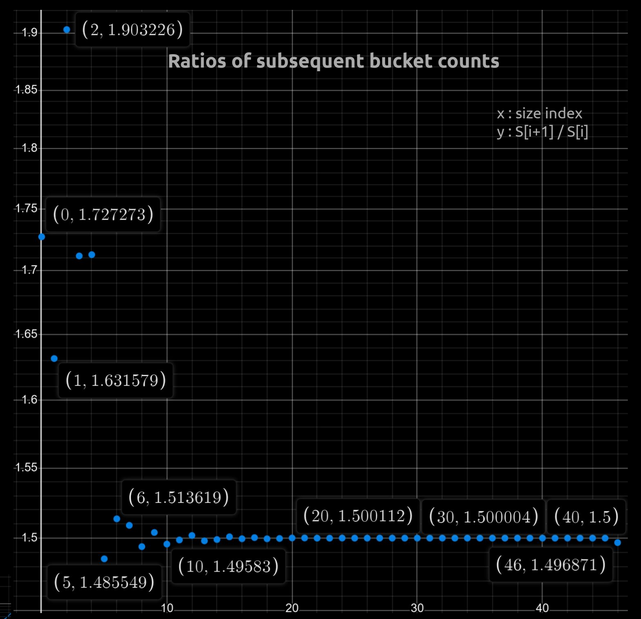
| | | |
|---|---|---|
| 8-bytes | key | pointer to the key string |
| 8-bytes | val | pointer to the value string |
| 8-bytes | next | arena index for next |

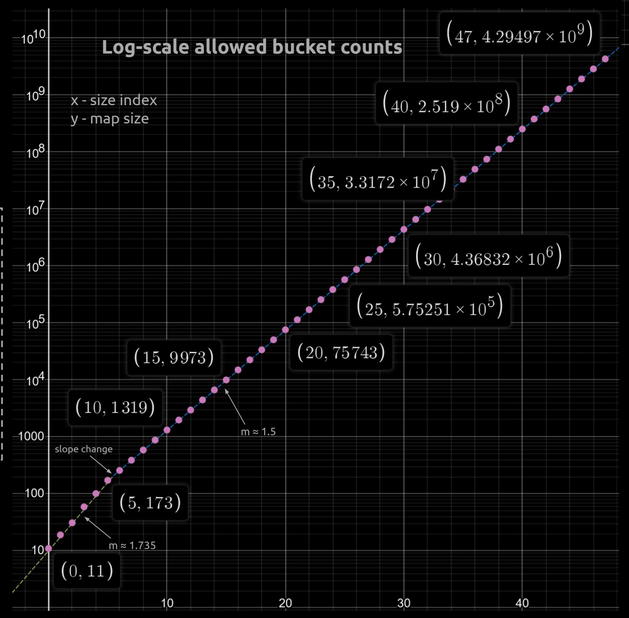
The map uses prime modulo because it allows growth factors other than 100% (compared to power-of-two-masking tables). 50% was chosen for medium and large maps, and ~74% was chosen for small maps where increases of larger steps will most likely decrease the overall work, while still not wasting too much memory in the worst case.

Below $2^{32}$ - 4 buckets, there are only 48 options for the number of buckets. For the bulk of the entries, they are the prime closest to $2 \cdot 1.5^{6+i}$ for $5 \le i \le 46$ (0-indexed i). the last element is $2^{32}$ - 5 (still prime), since the next value (4,303,945,123) does not fit in 32 bits. the first five values come from a geometric series that goes from 11 to 173 in 5 steps, which is a ratio of $\sqrt[5]{(173/11)}$, or around 1.735. Again, the values are the primes nearest to the values in the series. 11 was chosen to be the smallest option mostly arbitrarily, but kind of due to memory considerations. With 11 buckets as the minimum and 2 overflow slots at the minimum, a map object takes at least 368 bytes, which is a reasonable lower bound.

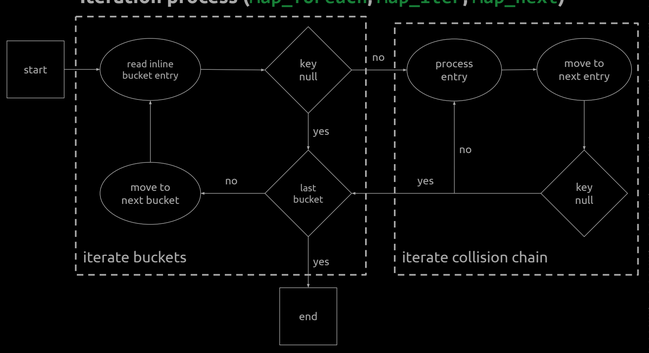
Above $2^{32}$ - 5 buckets, the only restriction on the map size is that it must be odd, since it would cost too much memory to store 64-bit primes, and finding the nearest prime to a given value is not trivial enough to be worth implementing. Also, at that scale, the number of buckets being prime stops mattering nearly as much as it does for with less buckets.

When selecting a bucket count via `map_size`, it will pick the value closest to the requested value. For example, 9973 buckets will be used if the requested size is anywhere in [8313, 12470]. If it is equally far between two options, it rounds up. For any existing valid size `m`, requesting a size of `m*3 >> 1` will move to the next size.




### Iteration process (`Map_foreach`, `Map_iter`, `Map_next`)




"process entry" either acts on entry using the block passed in to `Map_foreach`, or it is the point where `Map_iter` / `Map_next` returns the current iterator state, then the user processes it externally, and it moves to the next step in the diagram when the user resumes the iteration.

`Map_mresize` creates a new map, adds all the entries from the old map into the new map (rehash), and then shallow destroys the old map. If the new `m_cap` is too small, it will not recursively resize, it will just increase the overflow size, potentially leading to a lot of overflow entries compared to the bucket count. If it couldn't create the new map, or couldn't add all the entries, it will return the existing map without resizing. If the requested bucket count is the same as the existing map, it will *not* do a full rehash. The current implementation assumes the maps are not corrupted and do not have duplicate entries. If the map does have duplicate entries, it will not free any of the strings; this is because if it frees them, it is either correct or causes heap corruption, and if it does not free them, it is either correct or leaks memory. Leaking memory in the worst case is better than heap corruption in the worst case.

### `Map_merge` ownership models

| owned1 \ owned2 | true | false |
|---|---|---|
| true | **owning copy merge**<br>both maps own their strings<br>deep copy strings<br>free old on conflict<br>`other` is not cleared | **(special case) owning transfer merge**<br>both maps own their strings<br>shallow copy strings<br>free old on conflict<br>`other` is cleared |
| false | **non-owning copy merge**<br>only `dst` owns its strings<br>deep copy strings<br>free old on conflict<br>`other` is not cleared | **non-owning view merge**<br>neither map owns its strings<br>shallow copy strings<br>no frees on conflict<br>`other` is not cleared |

`owned1` is the ownership of `this`, and `owned2` is the ownership of `other`. `true` is owning and `false` is non-owning.

`this` is not allowed to be null.

if `other` is null or `other` is the same pointer as `this`, the function returns early.

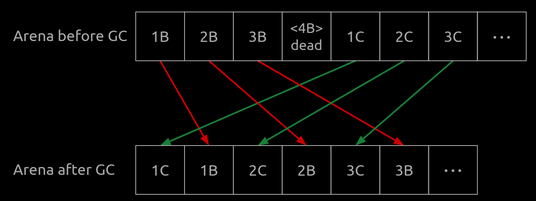
on conflicts, `other` wins (same as Python `dict.update`)

`owned1=false` with `owned2=true` is a special case because if interpreted literally, it will just be a memory leak, so instead it is an ownership transfer.

### `Map_gc` before/after example

**operations**
insert 1A, 2A, 3A, 4A (none collide)
insert 1B, 2B, 3B, 4B (all collide)
insert 1C, 2C, 3C, 4C (all collide)
delete 4A, 4B, 4C
run GC

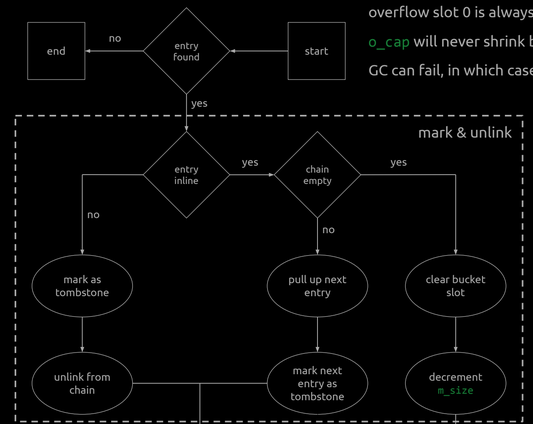
**ending bucket chains**
bucket 1: 1A → 1C → 1B
bucket 2: 2A → 2C → 2B
bucket 3: 3A → 3C → 3B
bucket 4: empty




Entries 1A, 2A, 3A, and 4A don't collide with anything, so they never get entered into the overflow arena; they stay in inline buckets. Insertion on collisions will prepend to the start of the overflow chain, but it will never evict the inline entry, so the entries after the first one in each bucket are stored in order of when they were most recently inserted. There is no <4C> shown because when the last element in the overflow is deleted, it is freed immediately, along with any consecutive tombstones before it. Since overflow uses a bump allocator, deleting the most recently inserted entry will always free it immediately (LIFO-esque). `Map_gc` preserves ordering within each chain.

The garbage collector is a tracing garbage collector, but it is not mark and sweep. It's main process is iterate and copy, and since the allocator doesn't know how to allocate something other than the end of the list, it must also compact at the same time.

### `Map_delete` core logic



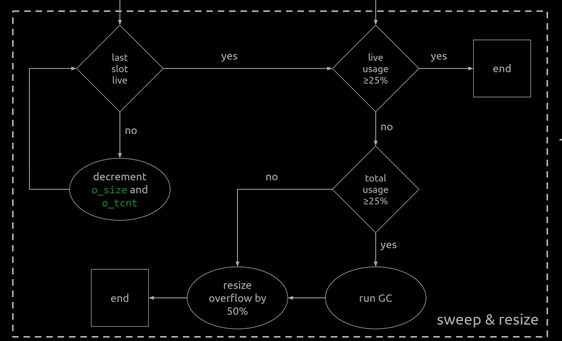

keys and values in owning maps are freed.

a "live" slot is one that cannot be freed.

overflow slot 0 is always live.

`o_cap` will never shrink below 4 (configurable).

GC can fail, in which case it will skip the resize.




`Map_set_raw` has a few cases: If the bucket is empty, or the key already exists in the map, it will just replace it the key and value with the new key and value (freeing the old ones if `owned` is true). If the entry goes to overflow and the insert would overwrite the buffer, it increases the overflow size by 50%. If the resize fails, it cancels the insert (freeing the new key and value if `owned` is true). It uses a bump allocator, so it does not look at where tombstones are, and will always allocate the next slot from the end. On overflow, it prepends to the overflow chain, but will never evict the inline entry, so if A, B, C, and D are inserted, the chain will look like this after each insert: A, A → B, A → C → B, A → D → C → B. This prepending logic was not made for any deep structural reason, but it allowed for more code reuse to prepend. When reached through `Map_set`, the overflow size increase will be unreachable; it only exists for if `Map_set_raw` is called directly. `Map_set` uses `Map_set_raw` for the underlying insert logic, but it adds on more complex resize logic which is generally useful, but sometimes adds unnecessary work (e.g. bulk inserts after a reserve).

** "map small" means <4096 buckets and "large size diff" means the overflow is 8 times larger than the map; this is kind of a rebalancing step.

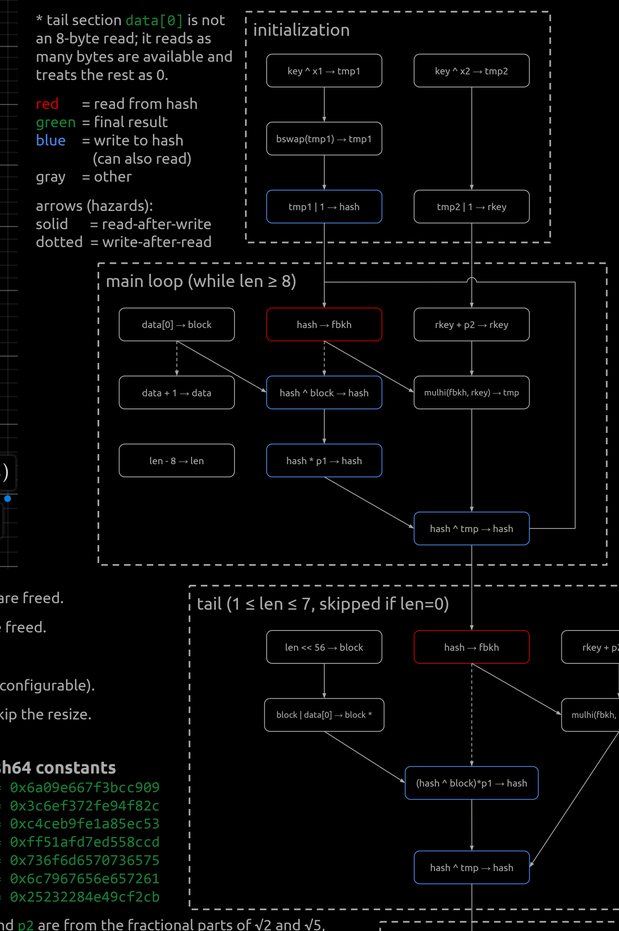
`data` is an array of unsigned 64-bit integers.

* tail section `data[0]` is not an 8-byte read; it reads as many bytes are available and treats the rest as 0.

red = read from hash
green = final result
blue = write to hash (can also read)
gray = other

arrows (hazards):
solid = read-after-write
dotted = write-after-read

### jhash64 data pipeline



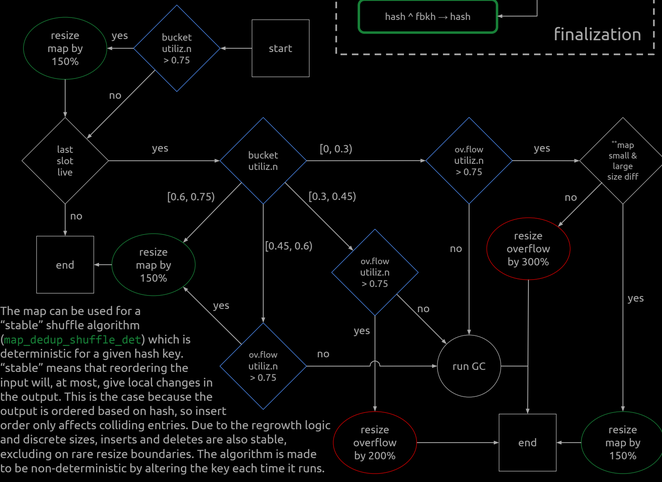

### jhash64 constants

```
p1 = 0x6a09e667f3bcc909
p2 = 0x3c6ef372fe94f82c
p3 = 0xc4ceb9fe1a85ec53
mx = 0xff51afd7ed558ccd
x1 = 0x736f6d6570736575
x2 = 0x6c7967656e657261
x3 = 0x25232284e49cf2cb
```

`p1` and `p2` are from the fractional parts of √2 and √5, respectively (modulo $2^{64}$). `p3` and `mx` come from MurmurHash3 fmix64. `x1` and `x2` are the SipHash initialization XOR constants. `x3` is a byte-swapped variant of the 64-bit FNV-1a starting hash. The constants in jhash128 are very similar, except `p3` and `mx` have random bytes appended to the start, and `x1` and `x2` also include the other two values from SipHash.

jhash64 and jhash128 are non-cryptographic as collisions can be generated fairly quickly. They pass most of the tests on SMHasher3, except for a few cases where the inputs are both short *and* sparse. These are being used to hash C-strings, so the inputs can't contain null bytes anyway, so it doesn't matter at all.

### `Map_set` resize logic




The map can be used for a "stable" shuffle algorithm (`map_dedup_shuffle_det`) which is deterministic for a given hash key. "stable" means that reordering the input will, at most, give local changes in the output. This is the case because the output is ordered based on hash, so insert order only affects colliding entries. Due to the regrowth logic and discrete sizes, inserts and deletes are also stable, excluding on rare resize boundaries. The algorithm is made to be non-deterministic by altering the key each time it runs.